

LETS AI - CLAUDECODE

Yogesh Haribhau Kulkarni

Outline

- 1 INTRODUCTION
- 2 GETTING STARTED
- 3 CONFIGURATION
- 4 USE CASES & DEMO
- 5 BEST PRACTICES
- 6 SUMMARY

Claude Code

Claude Code: Rapid Growth

- ▶ **4% of all public GitHub commits** are now authored by Claude Code as of February 2026 — around 135,000 commits every day.
- ▶ That represents a **42,896×** **growth** in 13 months since the research preview launched.
- ▶ **90% of Anthropic's own production code** is now AI-written, with Claude Code as the primary tool.
- ▶ **8 million+ MCP server downloads** by April 2025 — up from 100K in November 2024.
- ▶ Grew from a research preview into the **de-facto standard** for agentic terminal coding.
- ▶ Used by engineering teams at Stripe, Notion, and leading AI-native startups for production workflows.

What is Claude Code?

- ▶ Agentic coding assistant that lives in your **terminal**, built by Anthropic.
- ▶ **IDE Integration:** Works with VS Code, JetBrains (PyCharm, IntelliJ, WebStorm, etc.), and any terminal-supporting editor.
- ▶ **GitHub Integration:** Automate issues, PRs, and code review directly from the CLI.
- ▶ **Claude-Powered:** Backed by Anthropic's Claude models (Haiku, Sonnet, Opus) with access to a **200K token context window** (up to 1M in beta with Opus 4.6+).
 - ▶ *Enterprise:* AWS Bedrock, Google Vertex AI, Microsoft Foundry
 - ▶ *Direct:* Anthropic Console (API billing) or Claude Pro/Max subscription
- ▶ **Safety-First Design:** Fine-grained permission system; enterprise managed settings that cannot be bypassed.
- ▶ **Built by Anthropic:** First-party tool with direct access to frontier Claude models and the latest safety research.

Capabilities

Think of it as having a senior developer sitting next to you

- ▶ Writing and debugging code across an entire codebase
- ▶ Understanding complex project architectures
- ▶ Refactoring and optimization across multiple files
- ▶ Answering technical questions with project context
- ▶ Automating repetitive tasks end-to-end
- ▶ Managing git workflows: branches, commits, PRs
- ▶ Connecting to external services (databases, GitHub, Slack) via MCP

Key Features

- ▶ **Interactive REPL:** Terminal-native conversation loop; keyboard-driven with intuitive shortcuts.
- ▶ **Contextual Understanding:** Reads your entire repo, understands dependencies, structure, languages.
- ▶ **Agentic Workflows:** Plans changes, searches files, executes shell commands, modifies code autonomously.
- ▶ **Plan / Auto-Accept Modes:** Review proposed changes before applying; Shift+Tab cycles through modes.
- ▶ **Slash Commands:** /init, /help, /model, /compact, /cost, and more.
- ▶ **File References:** @file:path for scoped queries; paste images directly into the terminal.
- ▶ **Auto Memory:** Claude builds and updates CLAUDE.md automatically as it learns your project.
- ▶ **Extended Thinking:** Toggle with Alt+T for deep reasoning on complex problems.

Core Integration

- ▶ **MCP (Model Context Protocol):** Plug in external tools and data sources (GitHub, PostgreSQL, Slack, Jira, and 300+ more) to extend agent capabilities.
- ▶ **Hooks System:** 9 lifecycle events (PreToolUse, PostToolUse, Stop, SessionStart, etc.) for deterministic automation. Hooks always execute — unlike prompts.
- ▶ **Subagents:** Spawn specialist agents in isolated contexts (up to 10 in parallel); they return only summaries, preventing context bloat.
- ▶ **Skills:** Lazy-loaded SKILL.md folders that encode domain expertise. Loaded only when relevant — not on every prompt.
- ▶ **Plugins:** Installable bundles combining skills, subagents, hooks, and MCP configs into one shareable unit.
- ▶ **Permission System:** Fine-grained allow/deny/ask per tool; enterprise managed settings for compliance.

Claude Code vs Other Tools

Feature	Claude Code	OpenCode	Cursor/Copilot
First-party frontier model	✓	–	–
Hooks (3 handler types)	✓	–	Limited
Plugin system	✓	–	–
Subagents (up to 10 parallel)	✓	✓	–
MCP support	✓	✓	Limited
IDE integration (native)	✓	✓	✓
Local / offline models	–	✓	✓
Model-agnostic (75+ models)	–	✓	–
Open source	–	✓	–
1M token context	✓	–	–
Enterprise (Bedrock/Vertex)	✓	–	Limited

Claude Code is the **official** Anthropic tool; deepest integration with Claude models.

Installation: All Platforms

- ▶ **Quick Install (macOS / Linux):** `curl -fsSL https://claude.ai/install.sh`
bash—
- ▶ **macOS (Homebrew):** `brew install --cask claude-code`
- ▶ **Windows (PowerShell):** `irm https://claude.ai/install.ps1 | iex—`
- ▶ **Windows (CMD):**
`curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd`
- ▶ **npm (legacy, deprecated since v2.1.15):**
`npm install -g @anthropic-ai/claude-code`
- ▶ Migrate from npm to native binary: `claude install`
- ▶ Verify installation: `claude doctor`
- ▶ Key file locations (macOS/Linux):
 - ▶ Binary: `~/.local/bin/claude`
 - ▶ Global config: `~/.claude/settings.json`
 - ▶ Project config: `.claude/settings.json` (repo root)
 - ▶ Auth/state: `~/.claude.json`
 - ▶ Sessions: `~/.claude/sessions/`

Getting Started: Authentication

- ▶ **Launch:** Run `claude` in any project directory. The REPL loads; your first step is authentication.
- ▶ **Authenticate:** Run `claude auth login` → browser opens → sign in with Anthropic account.
- ▶ **Check status:** `claude auth status` shows current account, plan, and token expiry.
- ▶ **Config format:** JSON-based `settings.json`; supports project-level and global scopes.
- ▶ **First Step After Auth:** Run `/init` to ground Claude in your project — scans the codebase and creates `CLAUDE.md`.

Access Options in Claude Code

Claude Code supports three access paths.

Claude Pro / Max Subscription

- ▶ Single monthly plan covers both claude.ai web and CLI usage
- ▶ Predictable cost; no per-token billing surprises
- ▶ Includes rate-limited access to all Claude models

Anthropic Console (API Billing)

- ▶ Pay per token; no subscription; full API access
- ▶ Claude Code workspace auto-created at platform.claude.com
- ▶ Pricing (per 1M tokens, Sonnet 4.6): \$3.00 input / \$15.00 output
- ▶ Haiku 4.5: \$0.80 input / \$4.00 output (ideal for subagents)

Enterprise Platforms

- ▶ AWS Bedrock, Google Vertex AI, Microsoft Foundry
- ▶ Integrates with existing cloud billing and compliance controls
- ▶ Data residency options; no data retention by default

Enterprise Auth: Bedrock, Vertex, Foundry

Benefits for enterprise teams:

- ▶ Leverage existing cloud spend commitments and quotas
- ▶ Data stays within your cloud region (compliance)
- ▶ Works behind firewalls with proxy support
- ▶ Entra ID (Azure) or IAM (AWS) for identity management

Claude Code can route through your existing cloud infrastructure:

```
1 # AWS Bedrock
2 export CLAUDE_CODE_USE_BEDROCK=1
3 export AWS_REGION=us-east-1
4 export AWS_PROFILE=your-profile
5
6 # Google Vertex AI
7 export CLAUDE_CODE_USE_VERTEX=1
8 export CLOUD_ML_REGION=us-east5
9 export ANTHROPIC_VERTEX_PROJECT_ID=your-project
10
11 # Microsoft Foundry (Azure)
12 export CLAUDE_CODE_USE_FOUNDRY=1
13 export ANTHROPIC_FOUNDRY_RESOURCE=your-resource-name
14
15 # Corporate proxy
16 export HTTP_PROXY=https://your-proxy.com:8080
```

Plan vs Normal vs Auto-Accept Mode

Switch between them using Shift+Tab (cycles through all three).

Plan Mode:

- ▶ Read-only — cannot make changes to your code
- ▶ Analyzes and explores your codebase with full reasoning
- ▶ Writes a plan to `.claude/plans/` for your review
- ▶ Available tools: Read, Glob, Grep, WebSearch, WebFetch
- ▶ Perfect for: architecture decisions, complex refactors, unfamiliar codebases

Normal Mode (Default):

- ▶ Full access — can read, write, and modify files
- ▶ Prompts you to approve each file change
- ▶ Best for: careful development where you want per-edit review

Auto-Accept Mode:

- ▶ All edits applied without per-file confirmation
- ▶ Claude shows a running diff and cost in the status bar
- ▶ Interrupt at any time with `Ctrl+C`
- ▶ Best for: bulk scaffolding and trusted, well-specified tasks

Pro Tip: Always start complex features in Plan mode to review the approach, then switch to Normal or Auto-Accept for implementation.

Context Mentions & Image Support

- ▶ **@file:path** — focus the agent on a specific file:
@file:main.py fix the bug where user `input` isn't sanitized
- ▶ **@file:** without path — opens fuzzy file picker for the project.
- ▶ **Onboarding power:** Instead of manually grep-ing and reading files, just ask. Especially useful when exploring an unfamiliar codebase.
- ▶ **Image paste/drag-and-drop:** Paste screenshots or mockups directly into the terminal (Cmd+V on macOS). Claude sees the image and can implement UI from a design mockup, diagnose a browser error, or read a diagram.
- ▶ **Shift+Tab to Plan mode:** For complex changes, switch to Plan mode, describe the feature, and review the proposed approach before any code is written.
- ▶ **Alt+T for Extended Thinking:** Toggle deep reasoning mode for hard architectural or algorithmic problems.

Session Management

- ▶ **claude -c** (or **--continue**) — resumes the most recent session with full context.
- ▶ **claude --resume** — opens a session picker; select by name or number.
- ▶ **Name sessions:** Use **/rename auth-refactor** inside a session, or **claude -n "auth-refactor"** at launch for easy retrieval.
- ▶ **Fork a session:** **claude --resume <name> --fork-session** — branch to try an alternative approach without losing the original.
- ▶ **PR-linked sessions:** **claude --from-pr 42** — start a session linked to a specific pull request. Sessions auto-link when you create a PR via **gh** during a session.
- ▶ **/compact** — compresses the current session context to save tokens while preserving key decisions and code snippets.
- ▶ **/cost** — shows token usage and running session cost at any time.

(Ref: Anthropic, "Claude Code CLI Reference", docs.claude.com/en/docs/claude-code)

Non-Interactive Mode and CI/CD

- ▶ `claude -p` (print mode) — executes a single query and exits; ideal for scripting and CI.
- ▶ **Structured output:** `--output-format json` returns cost, duration, session ID, and result for parsing in pipelines.
- ▶ **Control agentic depth:** `--max-turns 10` prevents runaway loops; `--allowedTools` restricts what Claude can do.
- ▶ **GitHub Action:** Use `claude -p` in CI pipelines — auto-review PRs, generate changelogs, run AI-assisted linting.

```
1 # One-shot task
2 claude -p "Add docstrings to all public functions in src/"
3
4 # Structured output for scripts
5 claude -p "Count lines by file type" --output-format json
6
7 # Continue last session non-interactively
8 claude -c -p "Now add unit tests for those functions"
9
10 # In GitHub Actions:
11 claude -p "Review changed files for bugs: $CHANGED.
12   Post findings as a PR comment via GitHub MCP."
```

Configuration

Overview

- ▶ Claude Code is fully configurable at **two primary scopes**: User (~/.claude/) and Project (.claude/ in repo root).
- ▶ **Config priority** (highest → lowest): Enterprise managed → CLI flags → Local project .claude/settings.local.json → Shared project .claude/settings.json → User ~/.claude/settings.json.
- ▶ **Subdirectories**: agents/, skills/, commands/, rules/, plans/.
- ▶ **Meanings**:
 - ▶ settings.json : Shared permission rules
 - ▶ settings.local.json : Personal permissions (gitignored)
 - ▶ rules : Auto-enforced contextual rules
 - ▶ commands : Slash commands for common workflows
 - ▶ agents : Specialized subagent personas
 - ▶ skills : Auto-triggered workflow checks
- ▶ **Schema-validated**: Config validated against json.schemastore.org/claude-code-settings.json; editors autocomplete fields.
- ▶ **Team sharing**: .claude/settings.json and .mcp.json are committed to Git; .claude/settings.local.json is personal (add to .gitignore).
- ▶ Each section below covers: what ships built-in → what you can customize → how to create your own.

The core problem: Context is a finite budget. Dumping everything into one

Customization Layers: The Mental Model

- ▶ **CLAUDE.md: “The Onboarding Doc”:** *Always-on*, repo-committed, team-shared. Tells *any* agent how this project works: stack, build commands, test steps, conventions. Auto-updated by Claude as it learns. Think of it as the README for AI.
- ▶ **Rules: “Standing Orders”:** *Always-on* for matched paths, injected into every session. Placed in `.claude/rules/` with glob patterns. Encode constraints near high-risk directories (`src/auth/`, `infra/`).
- ▶ Claude Code has five extension mechanisms: commands, skills, subagents, hooks, and plugins. **Commands** = what to do. **Skills** = how to do it. **Subagents** = who does it. **Hooks** = guardrails that always run. **Plugins** = distributable bundles of all the above.
- ▶ **Skills: “The On-Call Expert”:** *Lazy-loaded*: Claude reads only the name and description at startup, fetches full content only when the task is relevant. Prevents token waste on irrelevant instructions.
- ▶ **Subagents: “The Specialist Persona”:** Own model tier, tool set, and system prompt. Run in isolated contexts. Return only summaries to the parent — preventing context bloat.
- ▶ **Hooks: “The Guardrails”:** Shell/prompt/agent handlers that fire automatically on lifecycle events. They are *deterministic* — not probabilistic like prompts.

Customization Layers: When to Use What

The layering principle: Rules = what to always know. Skills = what to know when needed. Subagents = who does the work. Hooks = rules that always enforce.

- ▶ **Use CLAUDE.md when:** The instruction is *project-wide*, stable, and useful to every agent, every session: build steps, test commands, folder conventions, PR rules. Commit to Git; share with the team. Auto-generate with `/init`.
- ▶ **Use Rules when:** The instruction applies to specific paths or is always relevant but not in the main `CLAUDE.md`: directory constraints, security invariants, package-level conventions. Global rules in `~/ .claude/CLAUDE.md` stay personal.
- ▶ **Use Skills when:** The instruction is *task-specific* and only occasionally needed: a `pytest-patterns` skill for test generation, a `refactor-patterns` skill for code cleanup. Keeps context lean.
- ▶ **Use Custom Subagents when:** You need a *different set of capabilities*: a review subagent that can't write files, a debug subagent with only `bash+read`, a docs subagent using a premium model.
- ▶ **Use Commands (/) when:** You have a *repetitive prompt*: a parameterized template you type often. Commands invoke subagents, pass arguments, and save keystrokes.

Customization Layers: Summary Table

Layer	Loaded	Scope	Purpose
CLAUDE.md	Always	Project / Personal	Project context
Rules	Path-matched	Project / Personal	Constraints
Skills	On-demand	Any	Domain expertise
Subagents	When invoked	Session role	Specialist work
Commands	When typed	Project / Global	Shortcuts
Hooks	Automatically	Session	Deterministic rules
Plugins	On install	Project / Global	Distribution

CLAUDE.md

CLAUDE.md helps Claude Code understand: project structure, coding conventions, technology stack, build commands, and more.

```
1  ## Project: My App
2
3  ## Tech Stack
4  - Next.js 14 with App Router
5  - TypeScript, Tailwind CSS
6
7  ## Build Commands
8  - Install: 'npm install'
9  - Dev:      'npm run dev'
10 - Tests:    'npm test'
11 - Lint:      'npm run lint'
12
13 ## Conventions
14 - Use functional components with hooks
15 - Type hints on all exported functions
16 - No hardcoded secrets: always use env vars
17
18 ## File Map
19 - /app      - Next.js app router pages
20 - /components - Reusable UI components
21 - /lib      - Shared utilities
```

Rules: Persistent AI Instructions

- ▶ **What:** Markdown instructions injected into every LLM context matching the path pattern. Think of them as standing orders: coding style, constraints, security invariants. Auto-enforced contextual rules
- ▶ **Built-in bootstrap:** `/init` auto-generates `CLAUDE.md` by scanning your repo: languages, deps, folder structure, patterns.
- ▶ **Auto Memory:** Claude automatically appends learnings to `CLAUDE.md` during sessions. Use `/memory` to view or clear them.
- ▶ **Scope:**
 - ▶ *Project rules:* `./CLAUDE.md` — commit to Git; shared across team.
 - ▶ *Global rules:* `~/.claude/CLAUDE.md` — personal, not committed.
 - ▶ *Directory rules:* Nested `CLAUDE.md` files in subdirectories; loaded for work in that subtree.
 - ▶ *Rules files:* `.claude/rules/*.md` with paths: glob patterns for targeted loading.
- ▶ **Create custom:** Write any Markdown file, reference it in `settings.json`. No special syntax: plain instructions work.

```
1 # .claude/rules/auth-rules.md
  paths: ["src/auth/**", "tests/auth/**"]
3 NEVER store tokens in localStorage.
  Always use httpOnly cookies for auth sessions.
```


Subagents: Built-in & Custom

- ▶ **What:** Specialist agents that run in isolated contexts, do focused work, and return only summaries to the parent conversation.
- ▶ **Isolation benefit:** Subagent exploration does *not* bloat your main context; only the conclusion returns.
- ▶ **Built-in subagent types (auto-selected by Claude):**
 - ▶ Explore: read-only codebase navigator; fast file/pattern search.
 - ▶ Plan: architect mode; designs approach without modifying files.
 - ▶ General: full tool access; multi-step parallel tasks.
- ▶ **Common custom subagents:** review (read-only + glob), debug (bash + read), docs (write, no shell), specs (write markdown only).
- ▶ **Create custom:** Write a Markdown file in `.claude/agents/<n>.md`. Claude auto-routes to a subagent when its description matches the task.
- ▶ **Subagent frontmatter** fields: name, description, model, allowed-tools.
- ▶ **Parallel execution:** Up to 10 subagents can run simultaneously for parallel research or review tasks.

Built-in Tools

- ▶ **What:** Functions Claude calls to act on your codebase. Core tools are enabled by default.
- ▶ **Core Built-in Tools:**
 - ▶ Read: read file contents (supports line ranges)
 - ▶ Edit / Write: exact string replacement; create/overwrite files
 - ▶ Bash: execute shell commands (git, npm, pytest, etc.)
 - ▶ Glob: find files by pattern (**/*.py)
 - ▶ Grep: regex search across codebase
 - ▶ WebFetch / WebSearch: fetch URLs or search during sessions
 - ▶ TodoWrite: manage task lists for multi-step work
- ▶ **Restricting tools per subagent:** Use allowed-tools in subagent frontmatter to scope access. A review subagent with only Read, Glob, Grep cannot accidentally delete files.
- ▶ **MCP tools:** External MCP servers expose additional tools, accessible as /mcp__servername__toolname.

```
1 # Subagent with restricted tools:
2 ---
3 name: review
4 allowed-tools: Read, Grep, Glob
5 ---
```

Skills: Reusable Behavior Packages

- ▶ **What:** Markdown instruction folders loaded on-demand by Claude when the task context matches. Unlike `CLAUDE.md` (always loaded), skills are lazy: Claude picks them only when relevant.
- ▶ **Skills ship with Claude Code:** Anthropic publishes official skills for docx, pdf, pptx, xlsx creation at github.com/anthropics/claude-code — usable in any project.
- ▶ **Discovery locations** (all checked at startup):
 - ▶ `.claude/skills/<n>/SKILL.md`: project-local
 - ▶ `~/claude/skills/<n>/SKILL.md`: global
- ▶ **Name rules:** Lowercase alphanumeric + single hyphens; directory name matches name field.
- ▶ **Activation:** Claude reads only the description field at startup. If the task matches, Claude loads the full `SKILL.md`. No manual invocation needed.

```
1 # .claude/skills/pytest-patterns/SKILL.md
2 ---
3 name: pytest-patterns
4 description: >
5   Best practices for writing pytest tests for this
6   project. Use when generating or reviewing Python
7   unit tests for mychatbot.
```

Commands: Slash-Invokable Shortcuts

- ▶ **What:** Reusable prompt templates invoked with / in the REPL. Eliminate repetitive prompt writing for common workflows.
- ▶ **Built-in commands:**
 - ▶ /init: scan project, generate CLAUDE.md
 - ▶ /model: switch active model
 - ▶ /compact: summarize context to save tokens
 - ▶ /cost: show current session token usage and cost
 - ▶ /help: list all available commands (including custom)
 - ▶ /hooks: list all configured hooks
 - ▶ /memory: view or clear auto-generated memory
- ▶ **Create custom commands** as .claude/commands/<n>.md (project) or ~/.claude/commands/<n>.md (global).
- ▶ **\$ARGUMENTS:** Anything typed after /command-name is injected here: makes commands parameterized.
- ▶ **Team sharing:** Commit .claude/commands/ to Git: the whole team gets it.

```
1 # .claude/commands/test-gen.md
2 ---
3 description: Generate pytest tests for a source file
4 allowed-tools: Read, Edit, Write, Bash
5 ---
6 Generate comprehensive pytest tests for $ARGUMENTS
```

Permissions: Granular Action Control

- ▶ **What:** Per-tool access policy: allow (auto-run), ask (prompt user), or deny (block). Defaults: most tools allow; .env files and secrets/denied by default.
- ▶ **Scope:** Set in .claude/settings.json (project) or ~/.claude/settings.json (user). Subagent allowed-tools overrides project settings for that agent.
- ▶ **Enterprise managed:** /etc/claude-code/managed-settings.json — cannot be bypassed by any user or project config.
- ▶ **Wildcard patterns:** Bash(git:*) allows all git commands; Edit(src/**) allows edits only in src/.
- ▶ **Caution:** --dangerously-skip-permissions bypasses the entire permission system. Never use in shared, CI, or production environments; only for fully-trusted local automation where the permission overhead is unacceptable.

```
2 { "permissions": {  
4   "allow": [  
6     "Read", "Glob", "Grep",  
    "Bash(npm run:*)",  
    "Bash(git commit:*)",  
    "Edit(src/**)"
```

Hooks: Deterministic Automation

- ▶ **What:** Shell/prompt/agent handlers that execute automatically at specific lifecycle events. Unlike prompts, hooks *always* run — regardless of model behaviour.
- ▶ **Why:** Memory is probabilistic. Tooling is deterministic. Use hooks for anything that must always happen: auto-formatting, security checks, test runs.
- ▶ **9 lifecycle events:** PreToolUse, PostToolUse, Stop, SubagentStop, SessionStart, SessionEnd, UserPromptSubmit, Notification, PreCompact.
- ▶ **Three handler types:** command (shell script), prompt (Claude model evaluation), agent (subagent with tool access).
- ▶ **Exit codes:** 0 = proceed, 2 = block + send reason to Claude.

```
1 { "hooks": {  
2   "PostToolUse": [  
3     { "matcher": "Edit|Write",  
4       "hooks": [{ "type": "command",  
5         "command": "black \"$CLAUDE_FILE_PATHS\" 2>/dev/null || true"  
6       }]  
7   }  
8 ],  
9   "PreToolUse": [  
10    { "matcher": "Bash",  
11      "command": "echo 'PreToolUse: Bash'"
```

MCP Servers: External Tool Integration

- ▶ **What:** Model Context Protocol servers expose external tools to Claude Code. Any MCP-compatible server becomes a tool Claude can call natively.
- ▶ **Why:** Extends Claude Code beyond the filesystem: databases, APIs, cloud services, monitoring tools, internal systems.
- ▶ **Popular MCP servers:** GitHub, Slack, PostgreSQL, Playwright, Brave Search, Linear, Jira, Notion, Sentry, AWS, Docker.
- ▶ **First-class CLI:** `claude mcp add`, `claude mcp list`, `claude mcp remove` — no manual JSON editing needed.
- ▶ **Scope flags:** `--scope local` (default, personal), `--scope project` (shared via `.mcp.json`), `--scope user` (all your projects).
- ▶ **Tool naming:** MCP tools appear as `/mcp__servername__toolname` slash commands. Use `mcp__github` in permissions to control all tools from one server.
- ▶ **Build your own:** Any HTTP/stdio process implementing the MCP spec; SDKs in TypeScript, Python, Go. Use FastMCP for Python servers.

MCP Servers: Configuration

```
1 # Add via CLI (recommended):
  claude mcp add github \
3   --transport http \
   https://api.githubcopilot.com/mcp \
5   -H "Authorization: Bearer $GITHUB_TOKEN"

7 # Or edit .mcp.json directly (committed to repo):
  { "mcpServers": {
8     "github": {
9       "command": "npx",
10      "args": ["-y", "@modelcontextprotocol/server-github"],
11      "env": { "GITHUB_TOKEN": "$GITHUB_TOKEN" }
12    },
13    "postgres": {
14      "command": "npx",
15      "args": ["-y", "@benborla29/mcp-server-postgresql",
16              "postgresql://localhost/mydb"]
17    },
18    "sequential-thinking": {
19      "type": "stdio",
20      "command": "npx",
21      "args": ["-y",
22              "@modelcontextprotocol/server-sequential-thinking"]
23    }
24  }
25 }
```


Plugins: Distributable Workflow Bundles

- ▶ **What:** Installable packages that combine skills, subagents, hooks, commands, and MCP configs. Solve the “works-on-my-machine” problem for team tooling.
- ▶ **Why:** Instead of every team member manually configuring their `.claude/` folder, package everything once and share.
- ▶ **Plugin structure:** A folder containing `.claude-plugin/plugin.json` + optional `commands/`, `agents/`, `skills/`, `hooks/`, `.mcp.json`, `scripts/`.
- ▶ **Namespacing:** Skills in plugins are namespaced `/pluginName:skillName` to avoid collisions.
- ▶ **Install / share:**
 - ▶ Local: `claude plugin add --path ./my-plugin`
 - ▶ Marketplace: `claude plugin install <name>`
 - ▶ Auto-install for team: list in `.claude/settings.json`
- ▶ **Community marketplace:** Growing library of open-source plugins at github.com/hesreallyhim/awesome-claude-code.

```
2 # .claude-plugin/plugin.json
3 { "name": "my-team-tools",
4   "version": "1.0.0",
5   "description": "Shared dev workflow for our team",
```

Models: Tier & Configuration

- ▶ **Three tiers**, each with a different cost-quality tradeoff:
 - ▶ **Haiku 4.5**: fastest, cheapest; ideal for subagents doing exploration and simple edits.
 - ▶ **Sonnet 4.6**: balanced default; covers most daily coding and refactoring tasks.
 - ▶ **Opus 4.6**: most capable; use for architecture design, complex reasoning, security review.
- ▶ Choose models with **long context window** and **tool use (function calling)** support.
- ▶ **Switch model** in REPL: `/model claude-opus-4-6`. Or `--model` CLI flag at launch.
- ▶ **Configure in .claude/settings.json**: `"model": "claude-sonnet-4-6"`.
- ▶ **Per-subagent model**: Each custom subagent can specify its own model tier — e.g., debug uses Haiku for fast file search; review uses Opus for thoroughness.
- ▶ **Extended thinking**: Available on Sonnet and Opus; toggle with Alt+T or include in subagent definition.
- ▶ **Context window**: 200K tokens standard; up to **1M tokens** with Opus 4.6+ (beta).

Models: Configuration Example

```
1 # .claude/settings.json
2 {
3   "$schema":
4     "https://json.schemastore.org/claude-code-settings.json",
5   "model": "claude-sonnet-4-6",
6   "permissions": {
7     "allow": ["Read", "Bash(npm run:*)", "Edit(src/**)"],
8     "deny": ["Read(.env*)", "Bash(rm -rf:*)"]
9   },
10  "env": {
11    "NODE_ENV": "development"
12  }
13 }
14
15 # Per-subagent model override in .claude/agents/review.md:
16 ---
17 name: review
18 model: claude-opus-4-6
19 allowed-tools: Read, Grep, Glob
20 ---
21
22 # Switch model in session:
23 /model claude-opus-4-6
24
25 # Launch with specific model:
26 claude --model claude-haiku-4-5
```

Real-World Use Cases

► **Understanding a Codebase:**

Explain the authentication flow `in` this project Claude analyzes your files and explains how auth works with actual file references.

► **Adding a Feature:** Add a dark mode toggle to the settings page Claude finds the settings page, identifies the theme system, implements the toggle, and updates styles.

► **Debugging:** The login form `is not` submitting. Error: [paste] Claude traces the issue, checks relevant files, and suggests targeted fixes.

► **Multi-File Refactoring:** Rename, restructure, or migrate patterns across the entire codebase atomically.

► **Documentation Generation:** Auto-generate README, API docs, and inline comments from code context.

► **Test Generation:** Scaffold unit/integration tests with awareness of existing test patterns.

► **Security Auditing:** Identify unsanitized inputs, API key leaks, or injection vulnerabilities.

Demo

Initialize & Understand

- ▶ **Goal:** Show how Claude Code scans and understands a new project from scratch.
- ▶ **Setup:** Open terminal inside any Git repository.
- ▶ **Launch:** `$ claude`
- ▶ **Command:** `> /init`
- ▶ **What Happens:** Claude scans all local files, infers the stack (languages, deps, structure), and writes `CLAUDE.md` — the AI's instruction manual for this project.
- ▶ **Key Point:** Every subsequent interaction is grounded in this context — no hallucinated file paths or invented conventions.
- ▶ **Auto Memory:** As Claude works, it appends learnings (build commands, debugging insights, discovered patterns) to `CLAUDE.md` automatically.

Fix & Refactor

- ▶ **Goal:** Demonstrate AI-driven code modification on a real file.
- ▶ **Command:** > @file:main.py fix the bug where user `input` isn't `sanitized`
- ▶ **Agent Action:** Reads `main.py`, identifies the unsanitized input vector, proposes a targeted patch.
- ▶ **Review Flow:**
 - ▶ Press Shift+Tab → switch to **Plan** mode: inspect the proposed diff before anything touches disk.
 - ▶ Press Shift+Tab → switch to **Normal** mode: approve each file change individually.
 - ▶ Press Shift+Tab → switch to **Auto-Accept** mode: apply all changes automatically.
- ▶ **Key Point:** You stay in control — the agent proposes, you decide the approval level.

Tips

Tips for Beginners

- ▶ **Be Specific:** Don't say "Make it better" but say, "Refactor the `handleSubmit` function to use `async/await` and add error handling."
- ▶ **Provide Context:** Don't say "Fix the bug" but say, "The login form throws 'undefined is not a function' when clicking submit. The error occurs in `auth.ts` line 45."
- ▶ **Use Plan Mode First:** For complex features, press `Shift+Tab` to switch to Plan mode before implementation. Review the full approach before a single file is touched.
- ▶ **Leverage Image Support:** Paste screenshots or mockups into the terminal — Claude Code can understand images and implement UI from visual designs.
- ▶ **Use Extended Thinking for Hard Problems:** Press `Alt+T` to toggle deep reasoning mode for architectural decisions and complex algorithmic problems.
- ▶ **Create Project-Specific Configs:** Different projects need different settings. `CLAUDE.md` and `.claude/settings.json` live in the repo root and travel with the codebase.

1. CLAUDE.md is Execution Memory

- ▶ Your CLAUDE.md (and ~/.claude/CLAUDE.md globally) should define:
 - ▶ WHY — product purpose
 - ▶ WHAT — repo map and file ownership
 - ▶ HOW — non-negotiable rules and build steps
- ▶ Keep it short and authoritative.
- ▶ If it becomes a knowledge dump, Claude will lose track of critical constraints.
- ▶ Clear, structural guidance beats long explanations.
- ▶ **Let Claude maintain it:** Auto memory means Claude updates CLAUDE.md as it learns — just review and commit.

2. Turn Repeated Workflows into Commands

Why:

- ▶ Standardize code review, refactoring, and release workflows.
- ▶ Prevent prompt drift across sessions and teammates.
- ▶ Use \$ARGUMENTS for parameterized commands.
- ▶ Commit `.claude/commands/` to Git: the whole team benefits.
- ▶ If you repeat the same prompt twice, make it a command.
- ▶ Create reusable command files in:

```
2 .claude/commands/test-gen.md    # /test-gen <file>
  .claude/commands/review.md     # /review
  .claude/commands/changelog.md  # /changelog "last week"
```

3. Use Hooks for Deterministic Guardrails

- ▶ Memory is probabilistic. Hooks are deterministic.
- ▶ Use hooks (not prompts) for anything that *must* always happen.
- ▶ Examples:
 - ▶ Auto-run Black formatter after every file edit (PostToolUse)
 - ▶ Run tests before Claude stops responding (Stop hook)
 - ▶ Block `rm -rf` and `DROP TABLE` commands (PreToolUse)
 - ▶ Desktop notification when Claude needs your attention (Notification)
 - ▶ Load git context at session start (SessionStart)
- ▶ Deterministic rules belong in hooks, not prompts.

4. Use docs/ for Progressive Disclosure

- ▶ Use `@file:docs/architecture.md` to load only what is relevant for the current task.
- ▶ Smaller context improves reliability and reduces hallucination risk.
- ▶ Keep architecture, ADRs, and runbooks in `docs/`.
- ▶ Reference documentation — do not paste it wholesale into context.

```
1 docs/  
  architecture.md  
3 decisions/  
  runbooks/  
5 specs/
```

5. Add Rules Files Near Sharp Edges

- ▶ Place `.claude/rules/` files near high-risk directories.
- ▶ Use the `paths`: glob pattern to load rules only when Claude touches those files.
- ▶ Document invariants, constraints, and warnings.
- ▶ Context is loaded exactly where it matters — not on every prompt.

```
1 .claude/rules/auth-rules.md
   paths: ["src/auth/**"]
3   NEVER store tokens in localStorage.

5 .claude/rules/migration-rules.md
   paths: ["db/migrations/**"]
7   Always test rollback before committing a migration.

9 .claude/rules/infra-rules.md
   paths: ["infra/**", "terraform/**"]
11  Never modify prod resources without a plan review.
```

6. Structure for Machine Onboarding

- ▶ If a new engineer would be confused, Claude will be too.
- ▶ Use clear module boundaries:
 - ▶ `src/` — application code
 - ▶ `api/` — interface layer
 - ▶ `domain/` — business logic
 - ▶ `infra/` — infrastructure and deployment
- ▶ Keep naming consistent and nesting shallow.
- ▶ Add nested `CLAUDE.md` files in complex subdirectories to give Claude targeted context for that area.

7. Smaller Context Beats Larger Context

- ▶ More files in context does not mean better output.
- ▶ Large context dilutes signal and increases the risk of ignoring critical constraints.
- ▶ Provide:
 - ▶ Exact files via @file:
 - ▶ Exact constraints via rules files
 - ▶ Exact task definition with acceptance criteria
- ▶ Use /compact when context grows large to summarize older turns.
- ▶ Route exploration work to subagents: they run in clean contexts and return only summaries.
- ▶ Precision beats volume.

8. Document Architecture Decisions (ADRs)

- ▶ Claude Code cannot infer historical decisions.
- ▶ Document *why* technologies and boundaries were chosen.
- ▶ Reference ADRs in `CLAUDE.md` so they are loaded each session.
- ▶ Prevents Claude from “optimizing” your architecture away based on general preferences.
- ▶ The conversation *is* the ADR — use `claude --resume` to revisit decisions.

```
1 docs/decisions/  
  001-use-postgres-not-mongo.md  
3  002-use-fastapi-not-django.md  
  003-auth-via-httponly-cookies.md
```

9. Treat Commands as Versioned Assets

- ▶ Store reusable commands in your repository.
- ▶ Version and improve them over time alongside the codebase.
- ▶ Avoid copy-paste prompt drift across sessions and teammates.
- ▶ Commands encode the team's *process*, not just the AI's capabilities.
- ▶ Package them in a plugin for distribution beyond the current repo.

```
.claude/commands/  
2 test-gen.md      # /test-gen src/auth.py  
  review.md       # /review  
4 changelog.md    # /changelog "last week"  
  security-audit.md # /security-audit src/
```

10. Structure ; Prompting Skill

- ▶ Strong structure transforms Claude Code into a workspace-native engineer.
- ▶ Combine:
 - ▶ File-based rules (CLAUDE.md, .claude/rules/)
 - ▶ Reusable commands (.claude/commands/)
 - ▶ Automated guardrails (hooks)
 - ▶ Documented architecture (docs/decisions/)
 - ▶ Minimal scoped context (@file:, subagents)
 - ▶ Distributable bundles (plugins)
- ▶ Prompts are temporary. Structure is permanent.
- ▶ The team that builds great Claude Code configuration ships faster than the team that relies on clever prompting.

Summary

Why Use Claude Code?

- ▶ **Frontier Model Quality:** First-party access to Claude Sonnet and Opus — the same models powering Anthropic's own 90%-AI-written codebase.
- ▶ **Safety-First Design:** Built-in permission system, enterprise managed settings, and hooks that enforce rules deterministically rather than probabilistically.
- ▶ **Hooks Guarantee Execution:** Formatting, security checks, test runs — hooks always fire at lifecycle events. Prompts rely on the model; hooks do not.
- ▶ **Subagents Prevent Context Bloat:** Route exploration and specialized work to isolated agents; the main conversation stays clean and focused.
- ▶ **Enterprise-Ready:** AWS Bedrock, Google Vertex AI, Microsoft Foundry, corporate proxy support, managed settings, and data residency options.
- ▶ **Terminal-Native Speed:** Faster context-switch than IDE plugins; integrates directly into your existing shell workflow and CI/CD pipelines.
- ▶ **Plugin Ecosystem:** Growing community of distributable workflow packages. Build once, share everywhere.
- ▶ **Bottom Line:** Faster dev loop, deterministic guardrails, enterprise-grade safety, AI that works *with* your terminal and your team.

Quick Reference: Config Cheat Sheet

- ▶ **File locations** (P = project `.claude/`, G = global `~/.claude/`):
 - ▶ Project memory: P: `CLAUDE.md` · G: `~/.claude/CLAUDE.md`
 - ▶ Rules: P: `.claude/rules/<n>.md`
 - ▶ Subagents: P: `.claude/agents/<n>.md` · G: `~/.claude/agents/<n>.md`
 - ▶ Skills: P: `.claude/skills/<n>/SKILL.md` · G: `~/.claude/skills/<n>/SKILL.md`
 - ▶ Commands: P: `.claude/commands/<n>.md` · G: `~/.claude/commands/<n>.md`
 - ▶ Hooks: P: `.claude/settings.json` → `"hooks": { ... }`
 - ▶ MCP: P: `.mcp.json` (shared) or `claude mcp add`
 - ▶ Permissions: P: `.claude/settings.json` → `"permissions": { ... }`
 - ▶ Enterprise: `/etc/claude-code/managed-settings.json`
- ▶ **Key CLI commands:**
 - ▶ `claude auth login`: authenticate / switch accounts
 - ▶ `/init`: generate `CLAUDE.md` for current project
 - ▶ `/model`: browse and switch models
 - ▶ Shift+Tab: cycle Plan / Normal / Auto-Accept modes
 - ▶ Alt+T: toggle extended thinking
 - ▶ `claude mcp add`: add an MCP server
 - ▶ `claude plugin add`: install a plugin
- ▶ **Priority order** (highest → lowest): Enterprise managed → CLI flags → `.claude/settings.local.json` → `.claude/settings.json` → `~/.claude/settings.json`.

The .claude/ Folder: Persistent Project Memory

- ▶ Most developers treat Claude Code like ChatGPT — every session a blank slate. **This is wrong.**
- ▶ The .claude/ folder is a **persistent configuration layer** that lives in your repo and loads automatically every session.
- ▶ Think of it as your AI assistant's onboarding document — except it actually reads and follows it.
- ▶ A well-structured .claude/ folder has **six layers**, each solving a different problem:

(Ref: Himanshu Bhoir, "The 15-File Setup That Turned Claude Code Into My Development Team", Towards AI, Mar 2026)

```
1 CLAUDE.md           # Project instructions (auto-loaded)
  CLAUDE.local.md     # Personal overrides (gitignored)
3 .claude/
  settings.json       # Shared permission rules (allow + deny)
5  settings.local.json # Personal permissions (gitignored)
  rules/              # Auto-enforced contextual rules
7  commands/          # Slash commands for common workflows
  agents/             # Specialized subagent personas
9  skills/            # Auto-triggered workflow checks
```

Layer 1: Rules That Enforce Themselves

- ▶ Rules are **markdown files** that activate automatically based on **file path globs**.
- ▶ You don't invoke them. You don't remember them. *They just work*.
- ▶ The same Claude instance behaves differently depending on where you're working:
 - ▶ Edit a Python utility? PEP 8 enforcement activates automatically.
 - ▶ Touch an agent file? ADK-specific patterns layer on top.
- ▶ Like having multiple specialized reviewers built into one tool, with zero manual invocation.

```
1 # .claude/rules/code-style.md (scoped to **/*.py)
2 ---
3 paths:
4   - "**/*.py"
5 ---
6 # Python Code Style
7 - Follow PEP 8; type hints required on all signatures
8 - Use the project logger -- never print() or stdlib logging
9 - Pydantic models for all request/response validation
10 - Async functions for all FastAPI endpoints
```


Layer 2: The Deny Rules Nobody Talks About

- ▶ Most guides focus on what to *allow*. The more important half is what you **deny**.
- ▶ Deny rules mean you never have to hesitate before approving an action.
- ▶ `settings.json` is committed to the repo; `settings.local.json` is gitignored: team shares safety rules, developers keep personal overrides.
- ▶ **Result:** Claude cannot read `.env` files, cannot force push, cannot `rm -rf` anything.

Layer 2: Deny Rules, settings.json Example

```
// .claude/settings.json
2 {
  "permissions": {
4    "allow": [
      "Bash(pip install *)", "Bash(python *)",
6      "Bash(git status)", "Bash(git diff *)",
      "Bash(pytest *)", "Read", "Write", "Edit"
8    ],
    "deny": [
10     "Bash(rm -rf *)",
      "Bash(git push --force *)",
12     "Bash(git reset --hard *)",
      "Read(.env)", "Read(.env.*)"
14   ]
15 }
16 }
```

Layer 3: Subagents With Assigned Roles and Models

- ▶ Claude Code lets you define **specialized subagents** — each with their own system prompt, tool access, and *model*.
- ▶ Assign the right model to the right task: **Haiku** for fast exploration, **Sonnet** for code review.
- ▶ This optimizes for both **cost and quality** at the project configuration level.

```
1 # .claude/agents/explore-codebase.md
2 ---
3 name: explore-codebase
4 description: Answers "how does X work?" questions
5 model: haiku
6 allowed-tools: Read, Grep, Glob
7 ---
8 Trace full request flows. Reference specific files
9 and line numbers. Keep answers concise.
10
11 # .claude/agents/code-reviewer.md
12 ---
13 name: code-reviewer
14 description: Reviews code for bugs, security, patterns
15 model: sonnet
16 allowed-tools: Read, Grep, Glob, Bash
17 ---
18 Flag actual bugs and logic errors, not style nitpicks.
```

Layer 4: Commands That Cover the Entire SDLC

- ▶ Commands are slash commands you invoke manually (`/project:git`, `/project:fix`).
- ▶ Build a set that covers your full development cycle — version them in the repo.

Command	Description
<code>/git</code>	Conventional commits, MR creation, branch management
<code>/fix</code>	Trace error to root cause, apply minimal fix
<code>/explain</code>	Map request flow through codebase with file refs
<code>/refactor</code>	Safe restructuring with impact analysis
<code>/test</code>	Run pytest, analyze failures, optionally auto-fix
<code>/review</code>	Security-first code review with file:line references
<code>/deploy</code>	Pre-deployment checklist for target environment

- ▶ The `/fix` command: paste an error, it traces the execution path and suggests a *minimal* fix. No unrelated refactoring. No style suggestions. Just the fix.

Layer 5: Auto-Triggered Skills

- ▶ Skills fire **automatically** when matching conditions are met. No slash command needed.
- ▶ Identify what you always forget to check, and make it automatic.
- ▶ Example: a security review skill that triggers whenever API endpoints, config files, or Dockerfiles are modified.
 - ▶ Checks: no hardcoded secrets, Pydantic validation on inputs, no secrets in logs, non-root Docker user.
- ▶ **Zero discipline required.** The review happens whether you remember to ask for it or not.
- ▶ Other ideas to add:
 - ▶ Auto-documentation skill when docstrings change.
 - ▶ Dependency audit when `requirements.txt` is modified.
- ▶ **Minimum viable setup** — start with just three files and grow from there:
 1. `CLAUDE.md` — stack, conventions, what Claude should never do.
 2. `settings.json` — deny rules for destructive commands and `.env` reads.
 3. One rule file — your most common code style violation.

Resources

- ▶ GitHub: github.com/anthropics/claude-code
- ▶ Docs: code.claude.com/docs
- ▶ Overview: docs.claude.com/en/docs/claude-code/overview
- ▶ Awesome Claude Code (community plugins, skills, agents):
github.com/hesreallyhim/awesome-claude-code
- ▶ Claude Code How-To Guide: github.com/luongnv89/claude-howto
- ▶ Blake Crosley, “Claude Code CLI: The Complete Guide”,
blakecrosley.com, 2026.
- ▶ Muneeb Ahmad, “Claude Code Extensions Explained: Skills, MCP, Hooks, Subagents, Agent Teams & Plugins”, Medium, Mar 2026.
- ▶ Himanshu Bhoir, “The 15-File Setup That Turned Claude Code Into My Development Team”, Towards AI, Mar 2026.
- ▶ MCP Server Registry: github.com/modelcontextprotocol/servers